Dominio 3 spiegato da zero: configurazione e workflow di Claude Code (20%)

Claude Code è lo strumento con cui gli sviluppatori fanno lavorare Claude direttamente sul codice. Questo dominio riguarda come lo **configuri per un team**: quali istruzioni riceve, come organizzi le sue abilità, quando lo fai pianificare prima di agire, e come lo usi in automatico.

Il filo conduttore: **la configurazione è codice condiviso**. Va messa nel posto giusto, perché il posto determina chi la vede e quando si applica.

> **Prima di iniziare.** Questo dominio dà per scontati molti concetti del lavoro quotidiano di uno sviluppatore. Quando incontri → **Pre-corso** trovi il link alla spiegazione completa. Per questo dominio conviene aver letto almeno:
>
> - Repository e Git (cosa finisce nel repository e cosa resta sul tuo computer: è la chiave di metà delle domande);
> - Terminale, shell e CLI e Argomenti e flag;
> - File system, percorsi e glob (cosa significano `~`, `./` e `**/*.test.ts`);
> - Markdown e YAML e frontmatter;
> - CI/CD e pipeline.
>
> Per i termini specifici di Claude tieni aperto il Vocabolario.

---

1. Dove gira Claude Code

Claude Code si può usare da più posti: terminale, editor (VS Code, JetBrains), app desktop, browser, cellulare. Il **motore è lo stesso** ovunque. Quindi le istruzioni, le impostazioni, le abilità e i collegamenti MCP funzionano allo stesso modo in tutti questi ambienti.

→ Pre-corso: Terminale, shell e CLI. Nella versione da terminale, Claude Code si avvia scrivendo il comando `claude`.

Per le attività pianificate, invece, il posto cambia il comportamento:

Meccanismo	Dove viene eseguito	Quando serve
Routines (`/schedule`)	Nel cloud	Compiti periodici, anche con il computer spento
Attività pianificate desktop	Sul tuo computer	Compiti che devono toccare file locali
`/loop`	Nella sessione corrente	Ripetere un'operazione mentre stai lavorando

Domanda tipica d'esame: "dove viene eseguito un compito pianificato che deve funzionare anche a computer spento?". Risposta: nel cloud, cioè con le Routines.

Il ragionamento è semplice: un'attività che gira sul tuo computer può leggere i tuoi file, ma si ferma quando lo spegni; un'attività nel cloud gira sempre, ma non vede i file che hai solo in locale.

---

2. Il file `CLAUDE.md`: il foglio di istruzioni

Il file `CLAUDE.md` contiene istruzioni che Claude legge all'inizio di ogni sessione. È come un regolamento: "in questo progetto usiamo questo stile, questi comandi, queste regole". È scritto in linguaggio naturale, non in codice.

→ Pre-corso: Markdown. `CLAUDE.md` è un file di testo in formato Markdown: titoli con `#`, elenchi con `-`.

Un esempio di contenuto:

```
# Progetto Negozio online

## Comandi
- Avvia i test con `npm test`
- Controlla lo stile con `npm run lint`

## Convenzioni
- Codice in TypeScript, mai JavaScript puro
- Le date si salvano sempre in UTC
- Ogni nuova funzione ha il suo test nella stessa cartella
```

Ma **non ce n'è uno solo**. Ce ne sono diversi, a livelli, e vengono caricati dal più generale al più specifico:

Livello	File	Chi lo vede	Dove
1. Policy aziendale	gestito dall'amministratore	Tutti i dipendenti	Controllato dall'azienda
2. Utente	`~/.claude/CLAUDE.md`	Solo tu, in tutti i progetti	Sul tuo computer
3. Progetto	`./CLAUDE.md` o `./.claude/CLAUDE.md`	Tutto il team	Nel repository, via Git
4. Progetto personale	`./CLAUDE.local.md`	Solo tu, in quel progetto	Escluso da Git
5. Directory	`src/api/CLAUDE.md`	Chi lavora in quella cartella	Caricato solo lì

→ Pre-corso: File system, percorsi e glob. `~` è la tua cartella personale (vale per tutti i progetti); `./` è la cartella del progetto corrente.

Un'analogia: pensa a un'azienda. La policy aziendale vale per tutti. Il tuo quaderno personale vale per te in ogni ufficio. Il regolamento di un reparto vale per chi ci lavora. Le note che tieni su una singola commessa valgono solo per quella commessa.

Come scegliere il livello giusto

Fatti due domande: **chi** deve seguire questa istruzione? **Dove** vale?

Istruzione	Livello
"Nel progetto usiamo TypeScript e i test si lanciano con `npm test`"	Progetto (`./CLAUDE.md`)
"Rispondimi sempre in italiano" (preferenza tua, in ogni progetto)	Utente (`~/.claude/CLAUDE.md`)
"Il mio database di prova locale è sulla porta 5433" (solo tu, solo qui)	`CLAUDE.local.md`
"Nella cartella `src/api` gli errori si gestiscono così…"	Directory (`src/api/CLAUDE.md`)
"Mai inviare dati dei clienti a servizi esterni" (regola aziendale)	Policy aziendale

Due distinzioni da non confondere:

- **Progetto vs utente**: il `CLAUDE.md` di progetto lo vede e lo modifica il team. Quello utente è solo tuo.
- **`CLAUDE.md` vs `CLAUDE.local.md`**: entrambi restano sul tuo computer, ma `CLAUDE.local.md` è pensato per un singolo progetto e va escluso da Git, così non finisce nel repository.

→ Pre-corso: Repository e Git, in particolare il file `.gitignore` (cosa significa "escluso da Git").

Errore tipico d'esame: un collega nuovo non riceve le convenzioni del progetto. Causa probabile: le convenzioni sono state scritte nel `CLAUDE.md` **utente** di chi ha configurato il progetto, che resta sul suo computer. Vanno spostate nel `CLAUDE.md` di **progetto**.

Altri dettagli utili:

- **`@import`** permette di includere altri file dentro `CLAUDE.md`, per esempio `@docs/coding-standards.md`. Si può annidare fino a 5 livelli. Serve a non avere un unico file enorme: ogni argomento sta nel suo file.
- **`AGENTS.md`** è un file alternativo usato da altri agenti. Claude lo legge solo se nel progetto non c'è `CLAUDE.md`. Se ci sono entrambi, vince `CLAUDE.md`. Puoi anche far importare `AGENTS.md` dentro `CLAUDE.md`, così non duplichi le istruzioni.
- **Memoria automatica**: Claude può salvare appunti suoi in `~/.claude/projects/<progetto>/memory/MEMORY.md`. All'avvio ne legge le prime 200 righe o 25 KB.

L'approfondimento su `CLAUDE.md`, rules, skill e settings (sezione *Approfondimenti*) entra nei dettagli.

---

3. Regole per percorso: `.claude/rules/`

Immagina una convenzione che riguarda tutti i file di test, sparsi in dieci cartelle diverse. Potresti mettere un `CLAUDE.md` in ogni cartella, ma diventerebbe ingestibile.

La soluzione è **un unico file** dentro `.claude/rules/`, con un campo `paths:` che dice a quali file si applica, per esempio `**/*.test.ts`. La regola viene caricata solo quando Claude lavora su quei file. E si applica anche alle cartelle create in futuro.

→ Pre-corso: File system, percorsi e glob e YAML e frontmatter.

Come si legge `**/*.test.ts`: "qualunque file il cui nome finisce con `.test.ts`, in qualunque cartella e sottocartella". Le due stelle `**` significano "a qualunque profondità".

Un esempio di file `.claude/rules/test.md`:

```
---
paths:
  - "**/*.test.ts"
---

- Ogni test ha un nome che descrive il comportamento atteso
- Non usare dati reali dei clienti nei test
```

Regola pratica: senza `paths:` la regola è sempre caricata; con `paths:` viene caricata solo quando serve.

Perché "solo quando serve" è un vantaggio? Perché ogni istruzione caricata occupa spazio nel contesto (vedi il dominio 5). Le regole sui test non servono mentre Claude lavora sulla grafica.

---

4. Le skill: le ricette riutilizzabili

Una **skill** è una ricetta: un insieme di istruzioni che Claude può usare quando serve. Ad esempio, una skill `deploy` contiene i passi per pubblicare il sito.

La differenza con `CLAUDE.md`: `CLAUDE.md` viene letto **sempre**, all'inizio. Una skill viene caricata **solo quando serve**, cioè quando Claude (o tu) decide di usarla. Così le istruzioni specialistiche non occupano spazio quando non servono.

Come si crea e come si chiama

Una skill è una cartella `.claude/skills/nome/` con un file `SKILL.md`. Dentro, un blocco iniziale (il *frontmatter*) con alcuni campi che ne regolano il comportamento. Il campo più importante è **`description`**: Claude lo legge per decidere da solo quando usare la skill. Una descrizione vaga porta a usarla al momento sbagliato, o a non usarla mai.

```
---
name: revisione-sicurezza
description: Controlla il codice modificato alla ricerca di problemi di sicurezza
(password nel codice, query SQL costruite con testo dell'utente, permessi troppo
ampi). Usala quando l'utente chiede una revisione di sicurezza o prima di un
rilascio.
---

Istruzioni della skill…
```

È lo stesso principio delle descrizioni degli strumenti nel dominio 2: Claude sceglie leggendo la descrizione.

→ Pre-corso: YAML e frontmatter

Un tempo i comandi personalizzati stavano in `.claude/commands/`. Funzionano ancora, e vengono trattati come skill.

Quando due cose hanno lo stesso nome

Se due elementi si chiamano allo stesso modo, vince quello con priorità più alta: **aziendale** > **personale** > **progetto** > **plugin**.

I campi che servono

Campo	Cosa fa
`description`	Dice a Claude quando usarla
`disable-model-invocation: true`	Solo tu puoi lanciarla, con `/nome`. Claude non la usa da sola
`user-invocable: false`	La nasconde dal menu `/`, ma Claude può usarla come conoscenza di contesto
`allowed-tools`	Strumenti approvati mentre la skill è attiva
`context: fork`	Esegue la skill in un subagente isolato, senza sporcare la conversazione principale
`paths`	La carica solo quando lavori su file corrispondenti

Esempio d'esame: per un deploy in produzione, vuoi che Claude non lo lanci mai da solo. Serve `disable-model-invocation: true`.

Altri esempi di scelta:

Esigenza	Campo
Una skill che contiene solo conoscenze di contesto (per esempio il glossario del dominio aziendale), da non mostrare nel menu	`user-invocable: false`
Una skill che legge decine di file e non deve riempire la conversazione	`context: fork`
Una skill che deve poter eseguire i test senza chiedere conferma ogni volta	`allowed-tools`

La trappola più importante: la memoria delle skill

Il corpo di una skill entra nella conversazione **una volta sola**. Claude non lo rilegge nei turni successivi. Quindi se una skill dice "1. analizza, 2. verifica, 3. riepiloga", dopo dieci turni Claude può aver dimenticato il passo 2.

La regola: le skill devono contenere **istruzioni permanenti**, scritte come regole, non passaggi da eseguire una volta. Le regole valide per tutta la sessione vanno in `CLAUDE.md`.

L'iniezione dinamica: un comando prima della lettura

Dentro una skill puoi scrivere un comando tra backtick preceduto da un punto esclamativo, per esempio ``!`gh pr diff```. Questo comando viene eseguito **prima** che il testo arrivi a Claude, e il suo risultato entra nel testo. Così la skill riceve dati aggiornati al momento dell'invocazione, per esempio il diff della pull request appena aperta. Non è la stessa cosa che chiedere a Claude di eseguire il comando come secondo passo.

→ Pre-corso: GitHub (cos'è una pull request) e Repository e Git (cos'è un diff). `gh` è il programma da riga di comando di GitHub.

Un'analogia: è come un modulo prestampato in cui alcune caselle vengono riempite in automatico (data, numero di pratica) prima che il modulo arrivi sulla tua scrivania.

---

5. Plan mode: pensare prima di agire

La **modalità plan** è una delle modalità di permesso di Claude Code. In plan mode Claude esplora il codice, progetta un piano, te lo presenta e **aspetta la tua approvazione** prima di modificare qualsiasi file.

È come un architetto che fa il progetto prima di far partire i muratori.

Usala quando:

- le modifiche toccano più file;
- ci sono decisioni architetturali o più approcci validi;
- l'ambito è ampio o i requisiti non sono chiari.

Non serve quando: devi correggere un errore in un file, o la modifica è piccola e precisa. Passare dal piano per un refuso è uno spreco.

Richiesta	Modalità
"Correggi il refuso nel messaggio di errore del login"	Esecuzione diretta
"Aggiungi un campo `telefono` al modulo di registrazione" (un file, chiaro)	Esecuzione diretta
"Migra l'autenticazione da sessioni a token" (molti file, scelte da fare)	Plan
"Rendi più veloce il sito" (vago)	Plan

Le modalità di permesso

→ Pre-corso: Permessi, dove trovi anche le regole allow, ask e deny.

Le altre modalità di permesso sono `default`, `acceptEdits`, `auto`, `dontAsk` e `bypassPermissions`. Quest'ultima è la più permissiva.

In parole semplici, vanno da "chiedi conferma per quasi tutto" (`default`) a "fai tutto senza chiedere" (`bypassPermissions`). `acceptEdits` accetta automaticamente le modifiche ai file. Più la modalità è permissiva, più servono altri controlli (permessi `deny`, hook) per evitare danni.

---

6. I subagenti: assistenti con un compito preciso

Claude Code può delegare un compito a un **subagente**, un assistente con il proprio contesto separato. Il vantaggio: le letture lunghe e rumorose non riempiono la conversazione principale.

È lo stesso concetto del dominio 1 (il modello coordinatore/subagenti), applicato a Claude Code.

Subagenti integrati:

- `Explore`: sola lettura. Esplora la codebase e restituisce un riassunto. Ideale per capire un progetto grande senza consumare la memoria della conversazione.

- `Plan`: sola lettura. Fa la ricerca preliminare per un piano.
- `general-purpose`: esplora e agisce, con tutti gli strumenti.

Puoi anche crearne di tuoi, come file in `.claude/agents/`, con impostazioni come gli strumenti disponibili (`tools`), il modello da usare (`model`), la modalità di permesso (`permissionMode`) e l'isolamento (`isolation`).

Esempio: un subagente `revisore` con solo gli strumenti `Read`, `Grep` e `Glob`. Può leggere tutto, ma non può modificare nulla: è il **privilegio minimo** applicato.

→ Pre-corso: Permessi (privilegio minimo) e, per l'isolamento, Repository e Git (cos'è un worktree).

Due regole da ricordare:

- **Un subagente non eredita la conversazione.** È lo stesso principio del dominio 1.
- **I permessi scendono dal genitore.** Se la sessione principale è in `bypassPermissions`, il subagente non può cambiarla. Nel caso d'esame: un subagente con `permissionMode: plan` dentro una sessione `bypassPermissions` resta in `bypassPermissions`.

---

7. Perfezionare il lavoro passo dopo passo

Quattro tecniche da riconoscere:

- **Esempi concreti**: quando le istruzioni in prosa producono risultati incoerenti, mostra 2–3 coppie di input e output attesi.
- **TDD** (sviluppo guidato dai test): prima scrivi il test che descrive cosa deve fare il codice, poi fai eseguire, condividi l'errore, ripeti.
- **Intervista**: chiedi a Claude di farti domande prima di implementare. Utile quando i requisiti sono vaghi.
- **Problemi interdipendenti vs indipendenti**: se due problemi si toccano, descrivili in un unico messaggio, altrimenti le correzioni possono entrare in conflitto. Se sono indipendenti, affrontali uno alla volta.

→ Pre-corso: Test, lint e code review (cos'è un test) e Prompt di sistema e few-shot (perché gli esempi funzionano).

Perché il TDD funziona così bene con Claude? Perché il test è un criterio **oggettivo**: o passa o non passa. Claude può provare, leggere l'errore e correggersi da solo, senza che tu debba valutare ogni tentativo. È di nuovo il tema deterministico del dominio 1: un controllo automatico vale più di un "sembra giusto".

---

8. Usare Claude Code in automatico: CI/CD

La **CI/CD** è il sistema automatico che verifica e pubblica il codice ogni volta che cambia. Claude Code può girare lì dentro.

→ Pre-corso: CI/CD e pipeline, con un esempio completo di pipeline GitHub Actions.

Il flag indispensabile è `-p` (o `--print`): dice a Claude Code di lavorare in modo non interattivo, senza aspettare una persona. Senza `-p` la pipeline si blocca in attesa di input che non arriverà mai.

Un'analogia: è la differenza tra una telefonata (serve qualcuno che risponda) e una lettera (la scrivi, la spedisci, e il destinatario la legge quando arriva). In una pipeline non c'è nessuno al telefono.

Un esempio di comando:

```
claude -p "Rivedi le modifiche di questa pull request e segnala solo bug e
problemi di sicurezza" \
  --output-format json --max-turns 10
```

Altri flag da conoscere:

Flag	Cosa fa
`-p`	Modalità automatica, obbligatoria in CI
`--output-format json`	Restituisce un risultato leggibile da un programma
`--json-schema`	Verifica che il risultato finale rispetti uno schema
`--max-turns N`	Limite rigido di passaggi
`--max-budget-usd X`	Limite di spesa in dollari
`--permission-prompts none`	Nega automaticamente le operazioni che richiederebbero conferma, quando nessuno può rispondere

→ Pre-corso: JSON e JSON Schema. L'output in JSON serve perché il passo successivo della pipeline (per esempio pubblicare i commenti nella pull request) è un programma, non una persona.

Batch o sincrono?

→ Pre-corso: Sincrono, asincrono e batch

Qui il dominio si collega al dominio 4. Il **Batch** è una modalità in cui invii molte richieste insieme e ricevi i risultati più tardi. Costa il 50% in meno, ma può impiegare fino a 24 ore.

- **Sincrono** (risposta immediata): per i controlli bloccanti, per esempio prima di approvare una modifica. Lo sviluppatore aspetta.
- **Batch** (risposta differita): per report notturni o controlli periodici, dove nessuno aspetta.

I risultati del batch arrivano in ordine casuale, quindi per riconoscerli usi `custom_id`, un'etichetta che hai assegnato a ciascuna richiesta.

La domanda da farsi è sempre: **qualcuno sta aspettando?** Se uno sviluppatore non può unire la sua modifica finché la revisione non finisce, serve il sincrono. Se il report si legge domattina, il batch fa risparmiare metà della spesa.

Revisione indipendente

La sessione che ha scritto il codice è meno brava a trovarne gli errori, perché conserva il ragionamento che l'ha portata alle scelte: tende a difenderle. La revisione va fatta in una **sessione separata**, senza contesto condiviso. È come far revisionare un compito a un collega che non l'ha scritto.

---

9. Le sei cose da ricordare

1. `CLAUDE.md` di progetto è condiviso; quello utente e `CLAUDE.local.md` sono personali.
2. Una convenzione sparsa in molte cartelle va in `.claude/rules/` con `paths:`.
3. Plan mode per modifiche ampie; esecuzione diretta per quelle piccole.
4. In CI, `-p` è obbligatorio.
5. Batch per ciò che non è urgente, sincrono per ciò che blocca.
6. La revisione va fatta in una sessione indipendente.

Per capire meglio

Nelle domande d'esame, due parole chiave indicano spesso la risposta: "**condiviso**" (quindi `CLAUDE.md` di progetto o `.claude/rules/` invece del file personale) e "**senza che nessuno risponda**" (quindi `-p` o `--permission-prompts none`).

Tre ragionamenti tipici

"Le convenzioni sui test valgono per file sparsi in molte cartelle, anche future." Non un `CLAUDE.md` per cartella, ma un file in `.claude/rules/` con `paths: ["**/*.test.ts"]`.

"La pipeline di revisione resta bloccata e va in timeout." Claude Code sta aspettando una risposta che nessuno darà. Manca `-p` (ed eventualmente `--permission-prompts none`).

"Claude lancia il deploy da solo quando pensa che il lavoro sia finito." La skill di deploy deve avere `disable-model-invocation: true`, così parte solo se la invochi tu con `/deploy`.